

TP ARDUINO - SIGNAL SOS AVEC LEDS

LYCÉE PROFESSIONNEL - FILIÈRE CIEL

Durée: 4 heures

Classe: BAC PRO CIEL

Plateforme: Tinkercad

OBJECTIFS PÉDAGOGIQUES

Compétences Électronique

- ☐ Identifier les composants électroniques (LED, résistance, Arduino)
- ☐ Lire et comprendre un schéma électrique
- ☐ Calculer la valeur d'une résistance de limitation de courant
- ☐ Réaliser un montage sur breadboard
- ☐ Câbler correctement un circuit avec polarité (anode/cathode)
- ☐ Utiliser un multimètre (simulation Tinkercad)

Compétences Programmation

- ☐ Comprendre la structure d'un programme Arduino (setup/loop)
- ☐ Utiliser les fonctions pinMode(), digitalWrite(), delay()
- ☐ Créer et utiliser des fonctions personnalisées
- ☐ Déclarer et utiliser des constantes
- ☐ Debugger avec le moniteur série

Compétences Transversales

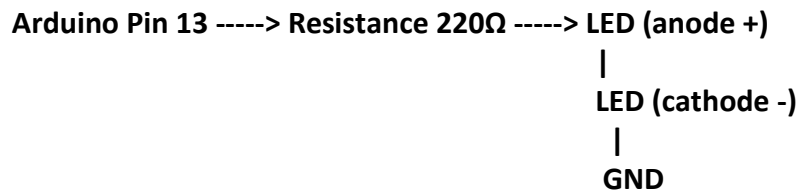
- ☐ Respecter un cahier des charges
- ☐ Travailler en autonomie
- ☐ Rechercher et corriger des erreurs
- ☐ Documenter son travail

PARTIE 1 : RAPPEL - ALLUMER UNE LED SIMPLE

Matériel nécessaire

- 1 Arduino Uno
- 1 LED rouge
- 1 résistance 220Ω (rouge-rouge-marron)
- Fils de connexion
- Breadboard

Schema de montage



RAPPEL IMPORTANT: - La **patte longue** de la LED = anode (+) → vers la résistance - La **patte courte** de la LED = cathode (-) → vers GND - La résistance protège la LED (limitation du courant)

Calcul de la résistance

Tension Arduino: 5V

Tension LED rouge: 2V

Courant LED souhaité: 20mA = 0,02A

Loi d'Ohm: $R = (U_{\text{arduino}} - U_{\text{LED}}) / I$

R = ?

→ On utilise(valeur normalisée supérieure)

Code Arduino simple

```
// Programme pour allumer une LED  
// Pin 13 de l'Arduino
```

```
void setup() {  
  // Configuration du pin 13 en sortie  
  pinMode(13, OUTPUT);  
}
```

```
void loop() {
```

```
// Allumer la LED
digitalWrite(13, HIGH);
delay(1000); // Attendre 1 seconde
```

```
// Éteindre la LED
digitalWrite(13, LOW);
delay(1000); // Attendre 1 seconde
}
```

Test du montage

1. Ouvrir Tinkercad et créer un nouveau circuit
 2. Réaliser le montage selon le schéma
 3. Copier et téléverser le code
 4. Lancer la simulation
 5. **Vérifier** : La LED doit clignoter (1s allumée, 1s éteinte)
-

PARTIE 2 : PROJET - SIGNAL SOS

Cahier des charges

Vous devez créer un système d'alarme qui envoie un signal de détresse SOS en code Morse avec 3 LEDs.

Code Morse SOS : - **S** = • • • (3 signaux courts) - **O** = — — — (3 signaux longs) - **S** = • • • (3 signaux courts)

Spécifications techniques : - Signal court (point) : 200 ms - Signal long (tiret) : 600 ms (3× le point) - Pause entre signaux : 200 ms - Pause entre lettres : 600 ms - Pause avant répétition : 1400 ms

Matériel nécessaire

- 1 Arduino Uno
- 3 LEDs rouges
- 3 résistances 220Ω
- Fils de connexion
- Breadboard
-

Schéma de câblage

Pin 8 ---> [R 220Ω] ---> LED1 ---> GND
Pin 9 ---> [R 220Ω] ---> LED2 ---> GND
Pin 10 ---> [R 220Ω] ---> LED3 ---> GND

Code à compléter

```
// =====  
// PROJET SOS - À COMPLÉTER  
// =====  
  
// PARTIE 1 : Définir les pins des LEDs  
const int led1 = 8;  
const int led2 = ____; // À COMPLÉTER  
const int led3 = ____; // À COMPLÉTER  
  
// PARTIE 2 : Définir les durées (en millisecondes)  
const int POINT = ____; // Durée d'un point  
const int TIRET = ____; // Durée d'un tiret (3× POINT)  
const int PAUSE_LETTRE = 600;  
const int PAUSE_MOT = 1400;  
  
void setup() {  
  // PARTIE 3 : Configurer les pins en sortie  
  pinMode(led1, OUTPUT);  
  pinMode(____, ____); // À COMPLÉTER pour led2  
  pinMode(____, ____); // À COMPLÉTER pour led3  
  
  Serial.begin(9600);  
  Serial.println("=== SIGNAL SOS ===");  
}  
  
void loop() {  
  // Lettre S (3 points)  
  afficherS();  
  delay(PAUSE_LETTRE);  
  
  // Lettre O (3 tirets)  
  afficherO();  
  delay(PAUSE_LETTRE);  
  
  // Lettre S (3 points)  
  afficherS();  
  delay(PAUSE_MOT);  
}  
  
// PARTIE 4 : Compléter la fonction afficherS()  
void afficherS() {  
  Serial.println("S : ...");  
  for(int i = 0; i < 3; i++) {  
    allumerLEDs();  
  }
```

```

    delay(____); // Durée d'un POINT
    eteindreLEDs();
    delay(POINT);
}
}

// PARTIE 5 : Compléter la fonction afficherO()
void afficherO() {
    Serial.println("O : ---");
    for(int i = 0; i < ____; i++) { // Combien de tirets ?
        allumerLEDs();
        delay(____); // Durée d'un TIRET
        eteindreLEDs();
        delay(POINT);
    }
}

```

```

// PARTIE 6 : Fonction pour allumer toutes les LEDs
void allumerLEDs() {
    digitalWrite(led1, ____); // HIGH ou LOW ?
    digitalWrite(led2, ____);
    digitalWrite(led3, ____);
}

```

```

// PARTIE 7 : Fonction pour éteindre toutes les LEDs
void eteindreLEDs() {
    digitalWrite(led1, ____); // HIGH ou LOW ?
    digitalWrite(led2, ____);
    digitalWrite(led3, ____);
}

```

PARTIE 3 : TRAVAIL À RÉALISER

Étape 1 : Montage (30 min)

1. Créer le circuit sur Tinkercad
2. Placer les 3 LEDs sur la breadboard
3. Ajouter les 3 résistances de 220Ω
4. Câbler selon le schéma
5. **Faire vérifier par le professeur ✓**

Étape 2 : Programmation (40 min)

1. Copier le code à compléter dans l'éditeur Arduino
2. Compléter les parties manquantes (PARTIE 1 à 7)

3. Vérifier la compilation (pas d'erreurs)
4. Téléverser le code
5. **Faire vérifier par le professeur ✓**

Étape 3 : Test et validation (20 min)

1. Lancer la simulation
2. Vérifier le signal SOS :
 - Les 3 LEDs clignotent ensemble
 - Respecte le code Morse (• • • — — — • • •)
 - Le cycle se répète correctement
3. Ouvrir le moniteur série et vérifier les messages
4. **Faire valider par le professeur ✓**

Étape 4 : Améliorations (30 min) - BONUS

Choisir au moins une amélioration:

Option A - Niveau 1 : Modifier la vitesse du signal (plus rapide ou plus lent)

Option B - Niveau 2 : Faire clignoter les LEDs en séquence (LED1 → LED2 → LED3) au lieu de toutes ensemble

Option C - Niveau 3 : Ajouter un bouton poussoir sur le pin 2 pour activer/désactiver le SOS

Option D - Niveau 4 : Créer un nouveau message Morse (HELP, CIEL, votre prénom...)

GRILLE D'ÉVALUATION

Compétences Électronique (/10 points)

Critère	Points	Acquis
Identification correcte des composants	1	☐
Respect de la polarité des LEDs (anode/cathode)	2	☐
Calcul de la résistance de limitation	2	☐
Câblage propre et organisé	2	☐
Montage fonctionnel sans erreur	3	☐

Compétences Programmation (/6 points)

Critère	Points	Acquis
Code complété correctement (PARTIES 1-7)	3	☐
Utilisation correcte des fonctions	1	☐
Respect des durées du code Morse	1	☐
Programme compile sans erreur	1	☐

Fonctionnement et Validation (/4 points)

Critère	Points	Acquis
Signal SOS correct et reconnaissable	2	☐
Répétition automatique du signal	1	☐
Moniteur série fonctionnel	1	☐

Bonus - Améliorations (/2 points)

Critère	Points	Acquis
Au moins une amélioration fonctionnelle	2	☐

TOTAL : /20 points

AIDE-MÉMOIRE

Fonctions Arduino de base

`pinMode(pin, mode);` // Configure un pin (INPUT ou OUTPUT)
`digitalWrite(pin, valeur);` // Écrit HIGH (5V) ou LOW (0V)
`delay(ms);` // Pause en millisecondes
`Serial.println(texte);` // Affiche dans le moniteur série

Code Morse international

A : •— B : —•• C : —•—
D : —•• E : • F : ••—
G : ——• H : •••• I : ••
J : •—•— K : —•— L : •—••
M : —— N : —• O : —•—
P : •—•— Q : —•— R : •—•
S : ••• T : — U : ••—
V : •••— W : •—• X : —••—
Y : —•— Z : —••

Durées en code Morse

- **Point (•)** = 1 unité de temps
- **Tiret (—)** = 3 unités de temps
- **Pause entre signaux** = 1 unité
- **Pause entre lettres** = 3 unités
- **Pause entre mots** = 7 unités

RESSOURCES COMPLÉMENTAIRES

Tinkercad : circuits.tinkercad.com

Documentation Arduino : arduino.cc/reference

Code Morse : wikipedia.org/wiki/Morse_code

Professeur : _____

Date : // _____

Classe : _____

Nom de l'élève : _____

Note obtenue : _____ /20